NETWORK ADMINISTRATION

Task Report

Lecture 3 Commands + AI Agent Automation

Student	**Farah Imad**
Course	**Network Administration (10686213)**
Semester	**First Semester 2026–2027**
Instructor	**Jehad Hamamreh**

1. Task Requirements

- Practice the commands from Lecture 3 on the Linux device and document the practical work with screenshots.
- Use an AI agent in the Linux environment to generate and execute commands.
- Make the AI-agent activity practical and creative, while documenting the commands, approvals and results.

2. Evidence Structure

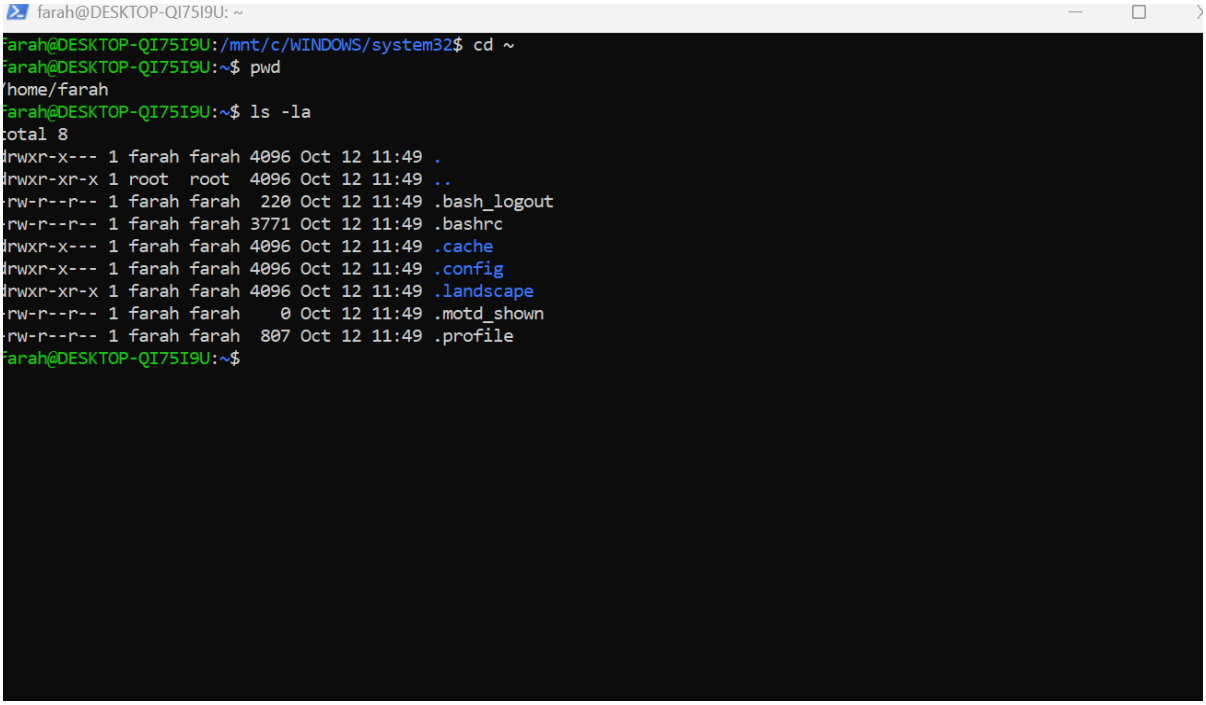
The report keeps every supplied screenshot as a separate evidence item. Each image is presented in full, followed immediately by a clear description of what is visible and what was demonstrated. No screenshot has been removed.

Section	Screenshots	Focus
Lecture 3 – Linux commands	1–20, 22–25	Navigation, manuals, files, text processing and vi.
AI-agent environment	21, 26–31	Ubuntu/WSL, packages, Node.js and Gemini CLI setup.
AI-agent Experiment 1	32–36	Creating and verifying hello.txt.
AI-agent Experiment 2	37–42	Creating mixed files and automatically organizing them by type.

3. Detailed Practical Evidence

3.1 Lecture 3 – Command Practice

Screenshot 1: pwd and ls -la



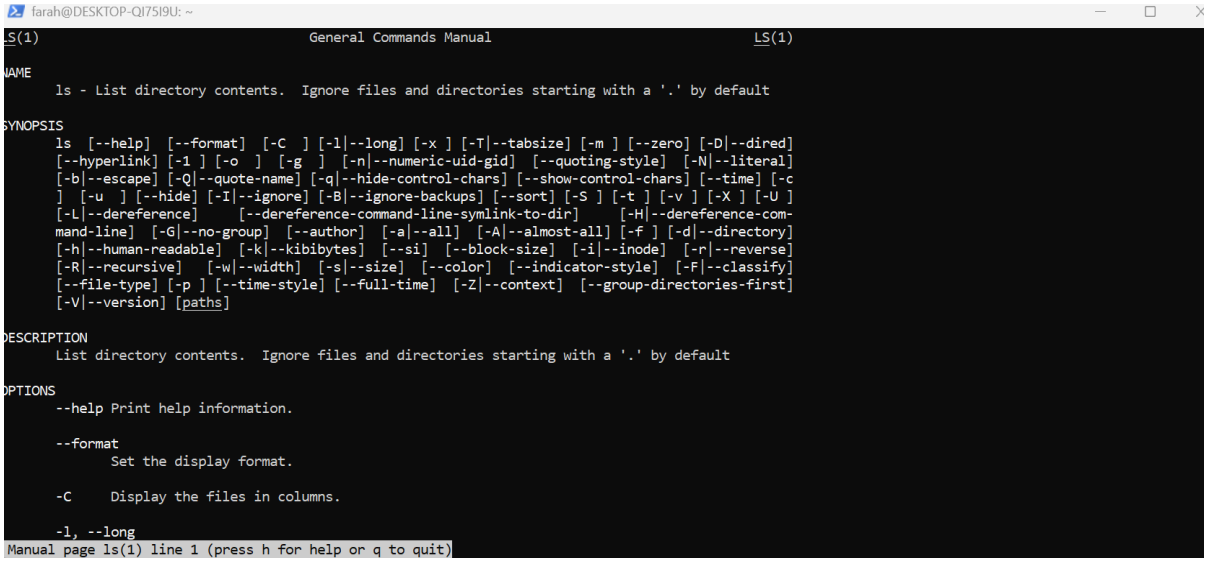
Description: The terminal starts in the user's home directory. `pwd` confirms the current path as /home/farah, and `ls -la` lists normal and hidden files with permissions, ownership and timestamps.

Screenshot 2: Directory navigation with cd and pwd



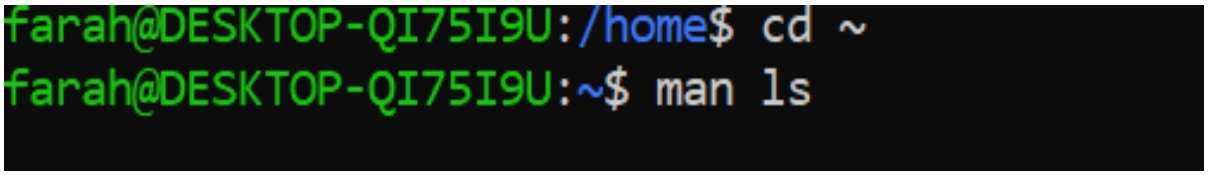
Description: The commands demonstrate moving to /usr/local/bin, checking the path with `pwd`, returning to the home directory with `cd ~`, and moving to the parent directory with `cd ..`.

Screenshot 3: Viewing the ls manual



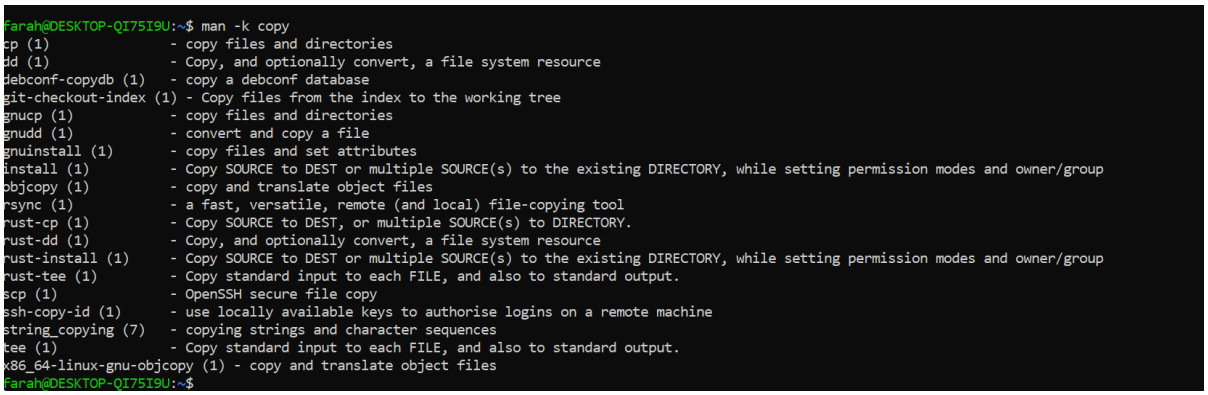
Description: The `man ls` command opens the manual page for ls and displays its syntax, options and description.

Screenshot 4: Opening the ls manual from the shell



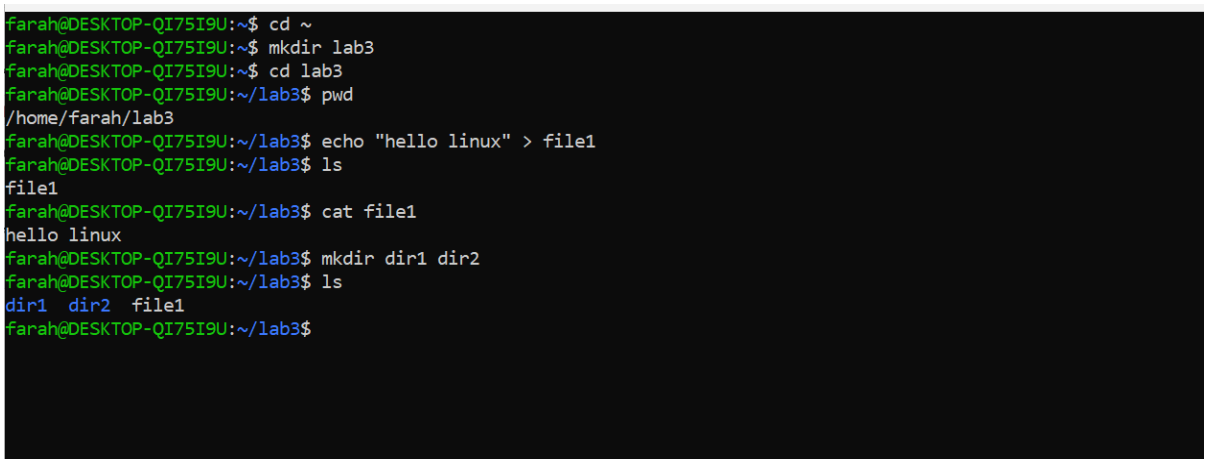
Description: This screenshot records the command used to invoke `man ls`, showing the transition from the shell prompt into the manual viewer.

Screenshot 5: Searching manual pages with man -k



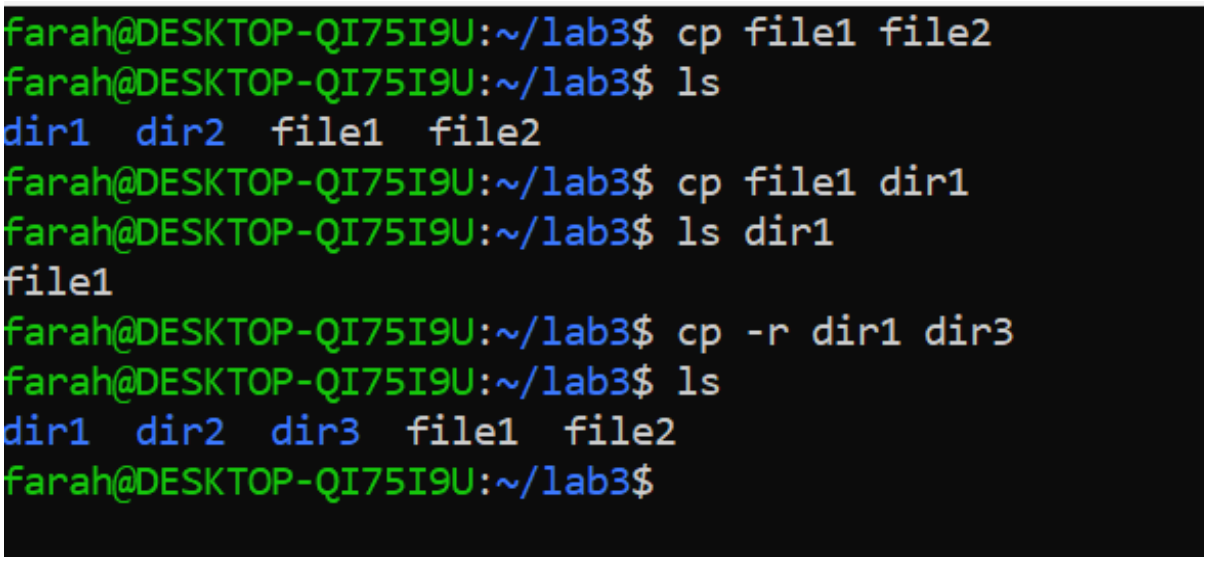
Description: The `man -k copy` search returns manual-page entries related to copying, demonstrating keyword-based manual lookup.

Screenshot 6: Creating a lab directory and files



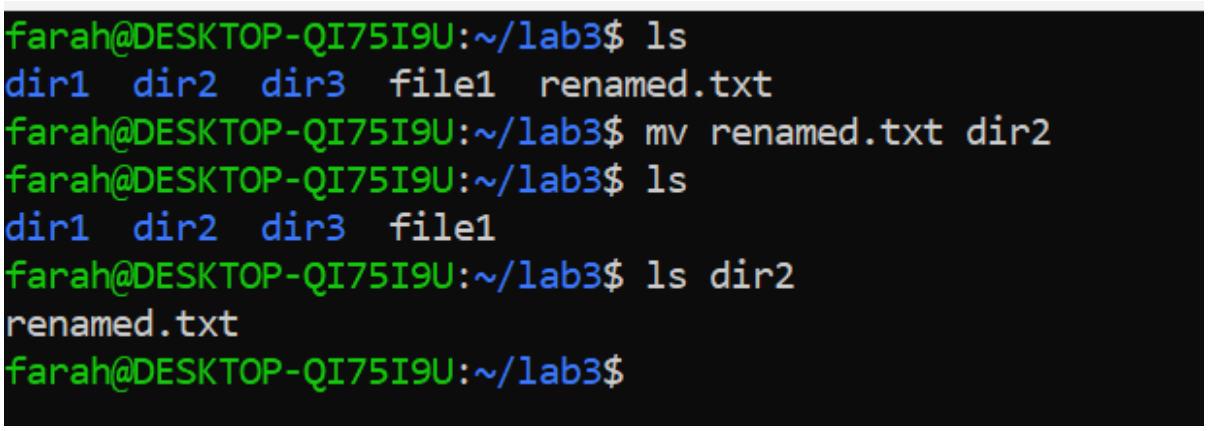
Description: The screenshot shows `mkdir lab3`, entering the directory, writing text into `file1` with echo, displaying it with `cat`, creating `dir1` and `dir2`, and checking the contents with ls.

Screenshot 7: Copying files and directories with cp



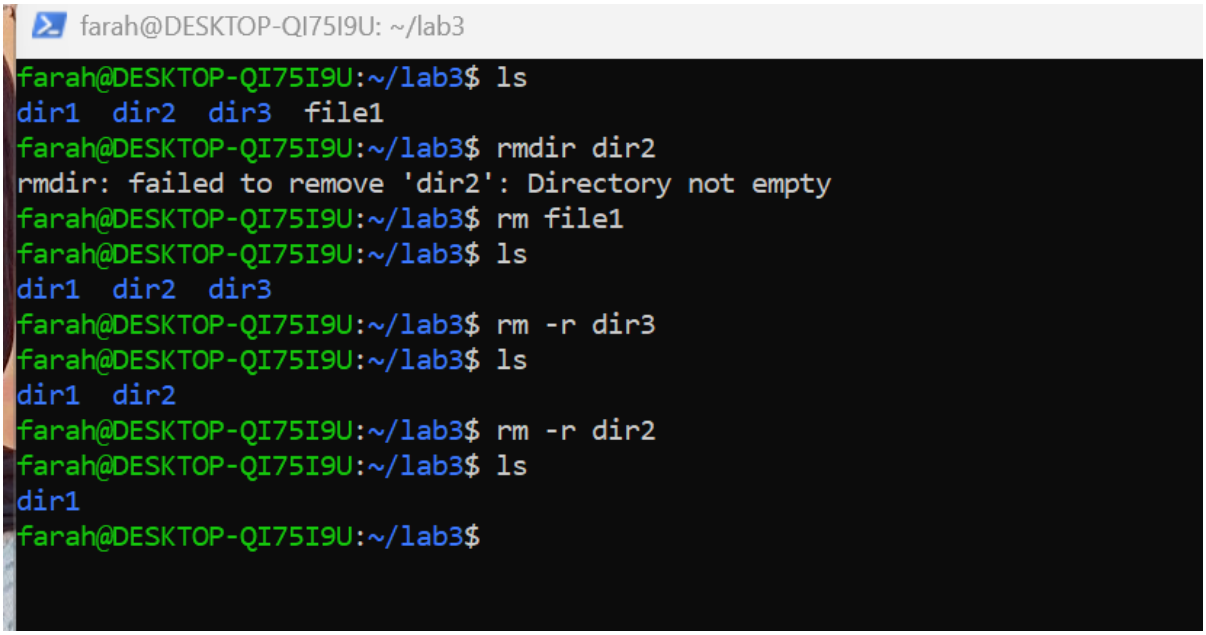
Description: The commands copy `file1` to `file2`, copy `file1` into `dir1`, and recursively copy `dir1` to `dir3` using `cp -r`.

Screenshot 8: Moving a file with mv



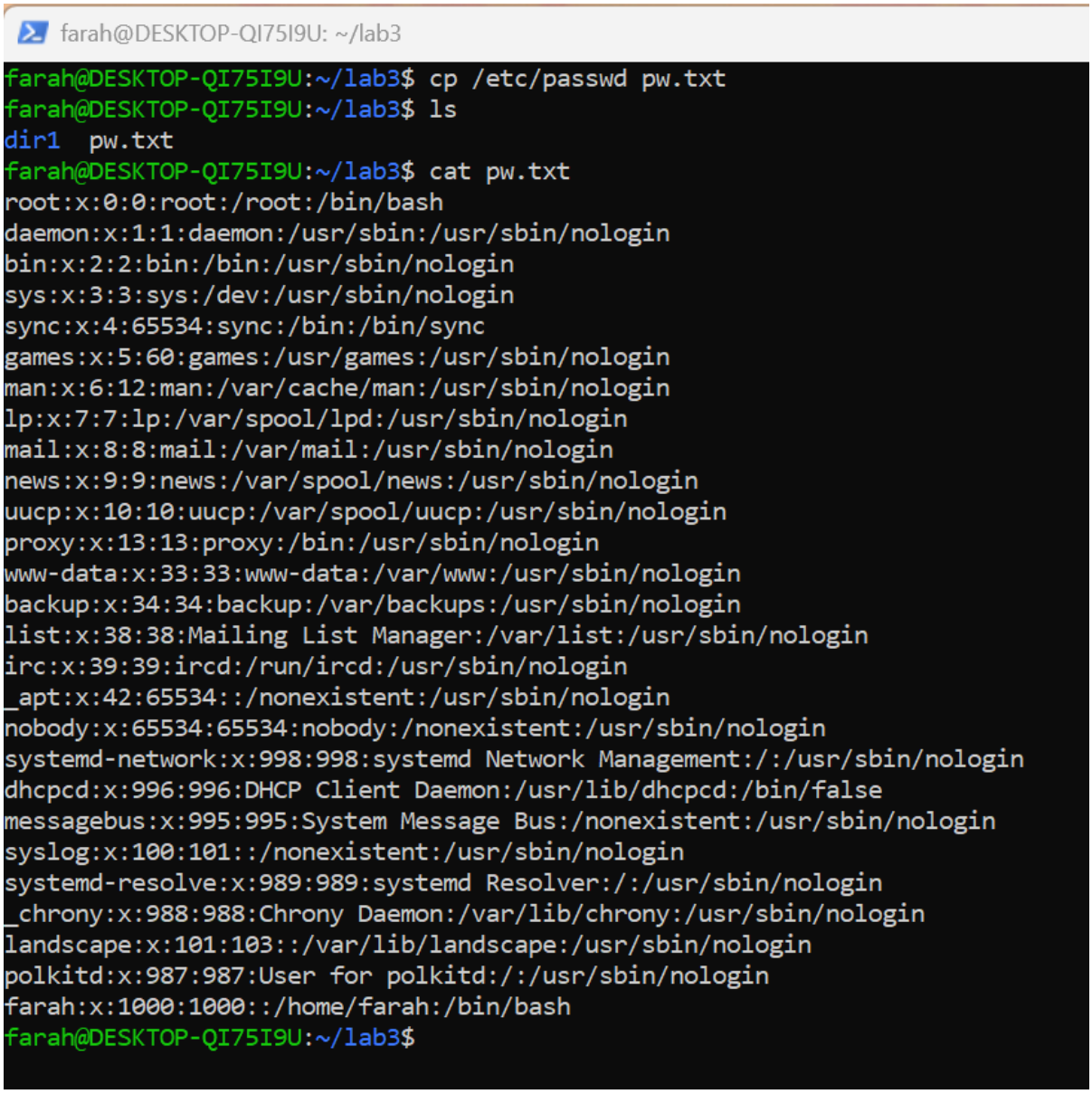
Description: The screenshot shows `renamed.txt` being moved into `dir2` and then verifies the destination with `ls dir2`.

Screenshot 9: Removing files and directories with rm



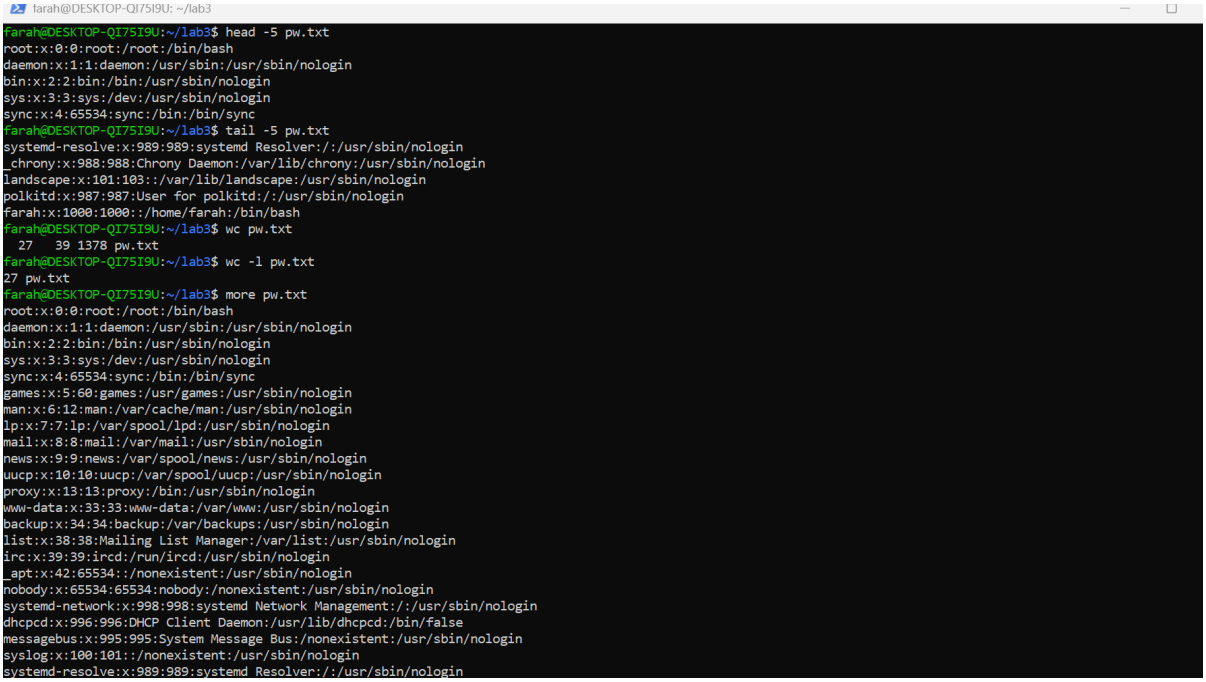
Description: The screenshot demonstrates removal of files and directories, including recursive removal with `rm -r`. It also shows the terminal response when a directory is not empty.

Screenshot 10: Copying /etc/passwd and displaying it



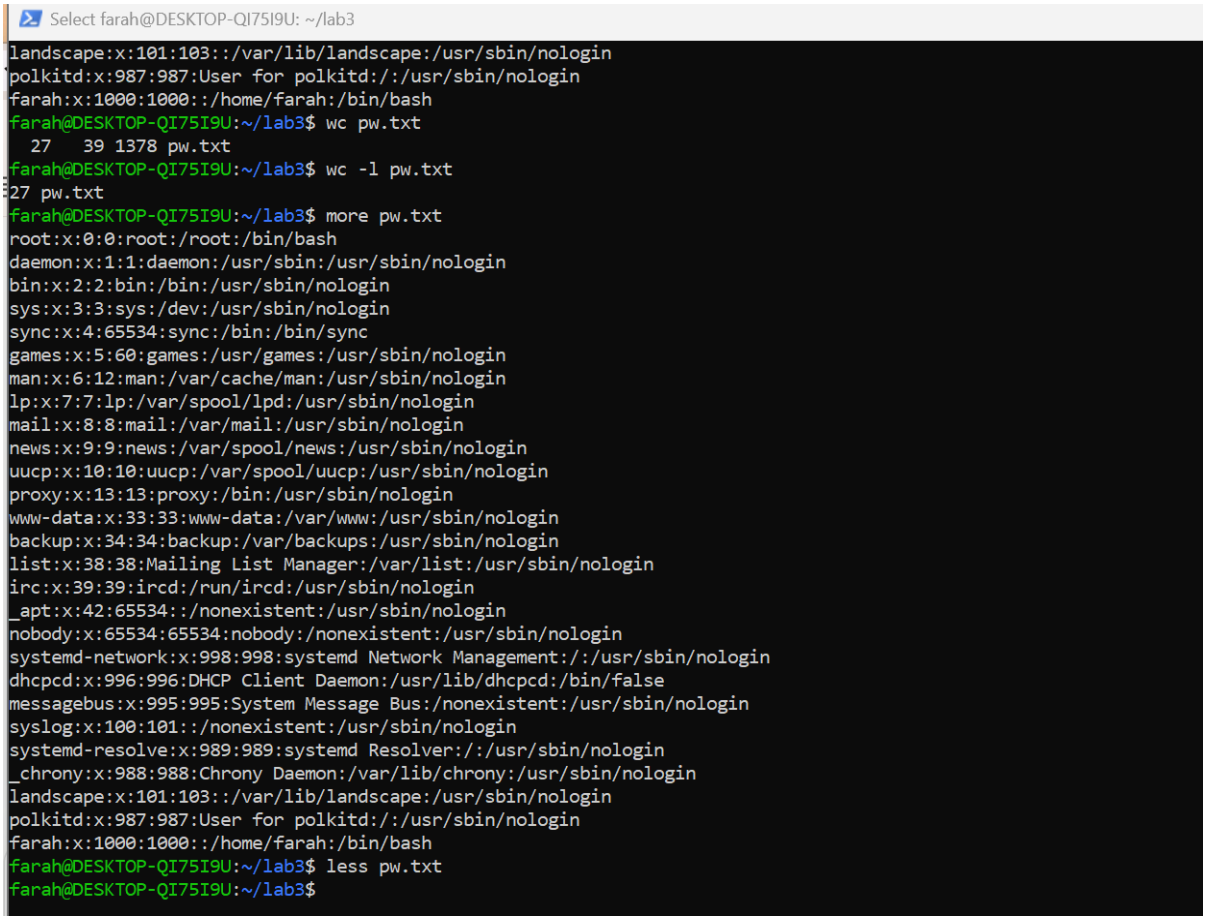
Description: The `/etc/passwd` file is copied into `pw.txt` and then displayed with `cat pw.txt`, providing a local file for the later text-processing exercises.

Screenshot 11: Viewing the beginning and end of a file



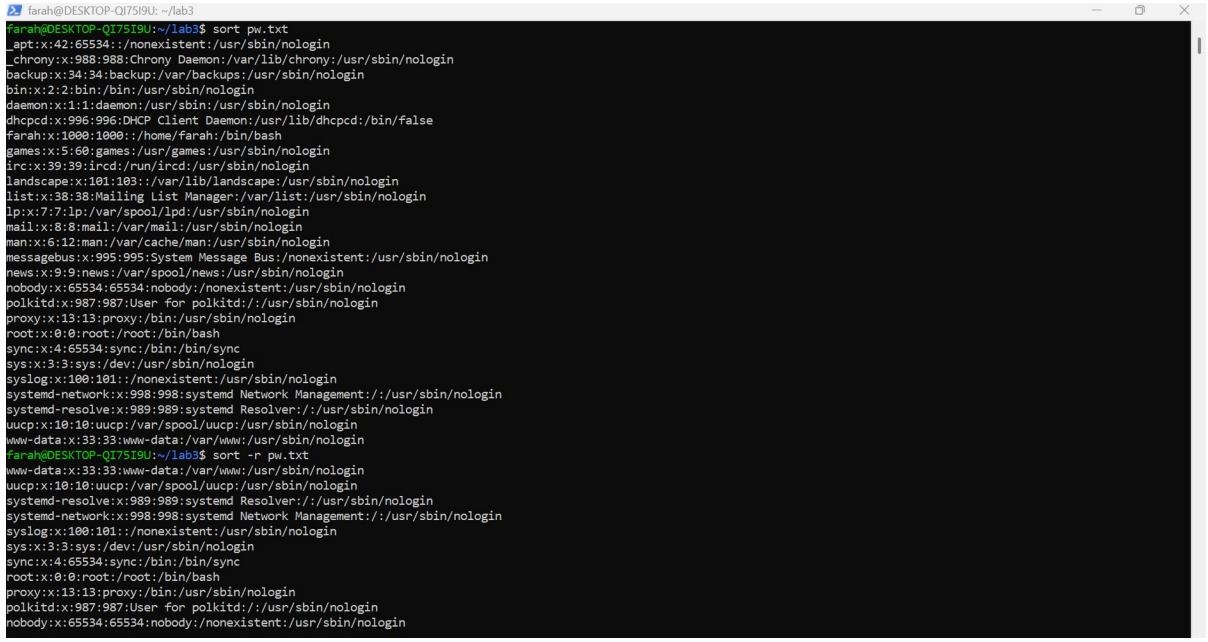
Description: The screenshot demonstrates `head -5 pw.txt` and `tail -5 pw.txt` to inspect the first and last five lines of the passwd file.

Screenshot 12: Counting and paging through a file



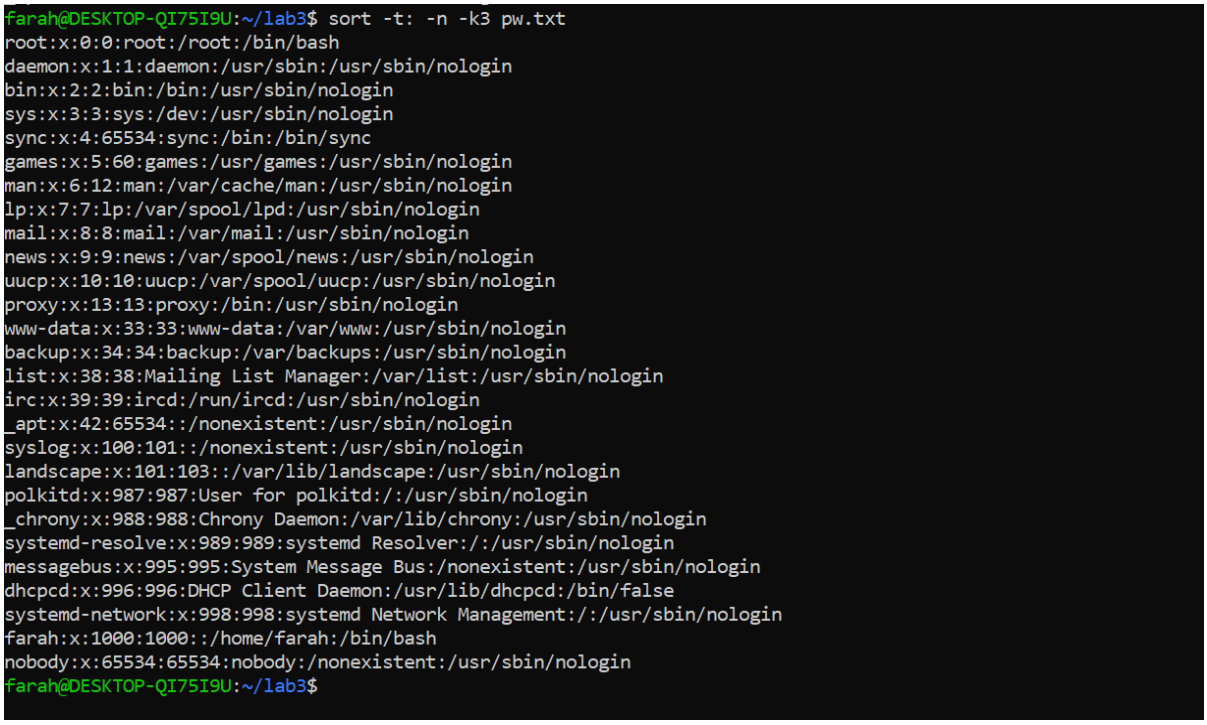
Description: The screenshot records file inspection using `wc -l` together with paged viewing commands such as `more`/`less`.

Screenshot 13: Sorting passwd data



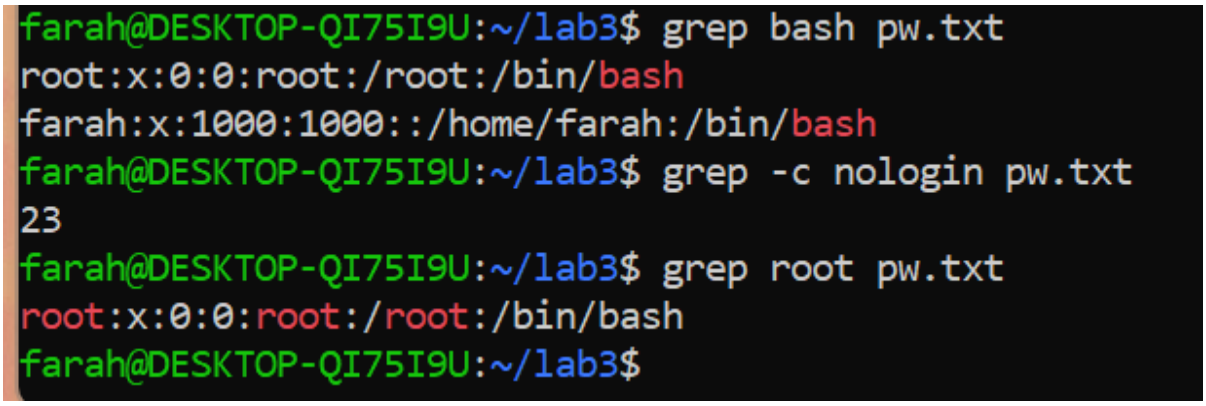
Description: The `sort pw.txt` command is used to sort the contents of the passwd file and inspect the resulting order.

Screenshot 14: Numeric sorting by a selected field



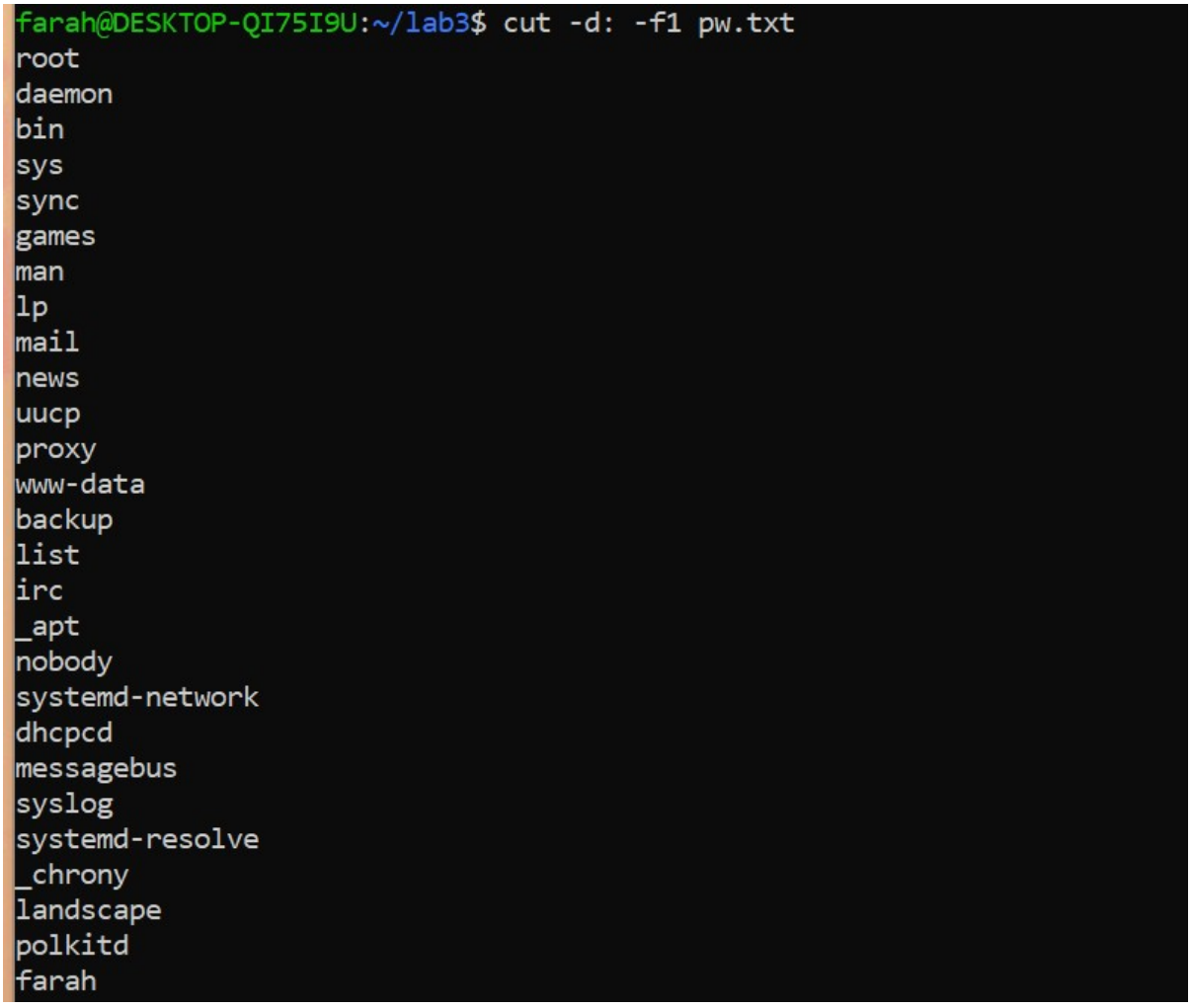
Description: The screenshot shows `sort -t: -n -k3 pw.txt`, using `:` as the field separator and sorting numerically by the third field.

Screenshot 15: Searching text with grep



Description: The commands search for `bash`, count `nologin` entries with `grep -c`, and search for `root` in `pw.txt`.

Screenshot 16: Extracting the first field with cut



Description: The command `cut -d: -f1 pw.txt` extracts the first colon-separated field, producing a list of usernames.

Screenshot 17: Extracting multiple fields with cut

```
farah@DESKTOP-QI75I9U:~/lab3$ cut -d: -f1,7 pw.txt
root:/bin/bash
daemon:/usr/sbin/nologin
bin:/usr/sbin/nologin
sys:/usr/sbin/nologin
sync:/bin/sync
games:/usr/sbin/nologin
man:/usr/sbin/nologin
lp:/usr/sbin/nologin
mail:/usr/sbin/nologin
news:/usr/sbin/nologin
uucp:/usr/sbin/nologin
proxy:/usr/sbin/nologin
www-data:/usr/sbin/nologin
backup:/usr/sbin/nologin
list:/usr/sbin/nologin
irc:/usr/sbin/nologin
_apt:/usr/sbin/nologin
nobody:/usr/sbin/nologin
systemd-network:/usr/sbin/nologin
dhcpcd:/bin/false
messagebus:/usr/sbin/nologin
syslog:/usr/sbin/nologin
systemd-resolve:/usr/sbin/nologin
_chrony:/usr/sbin/nologin
landscape:/usr/sbin/nologin
polkitd:/usr/sbin/nologin
farah:/bin/bash
farah@DESKTOP-QI75I9U:~/lab3$
```

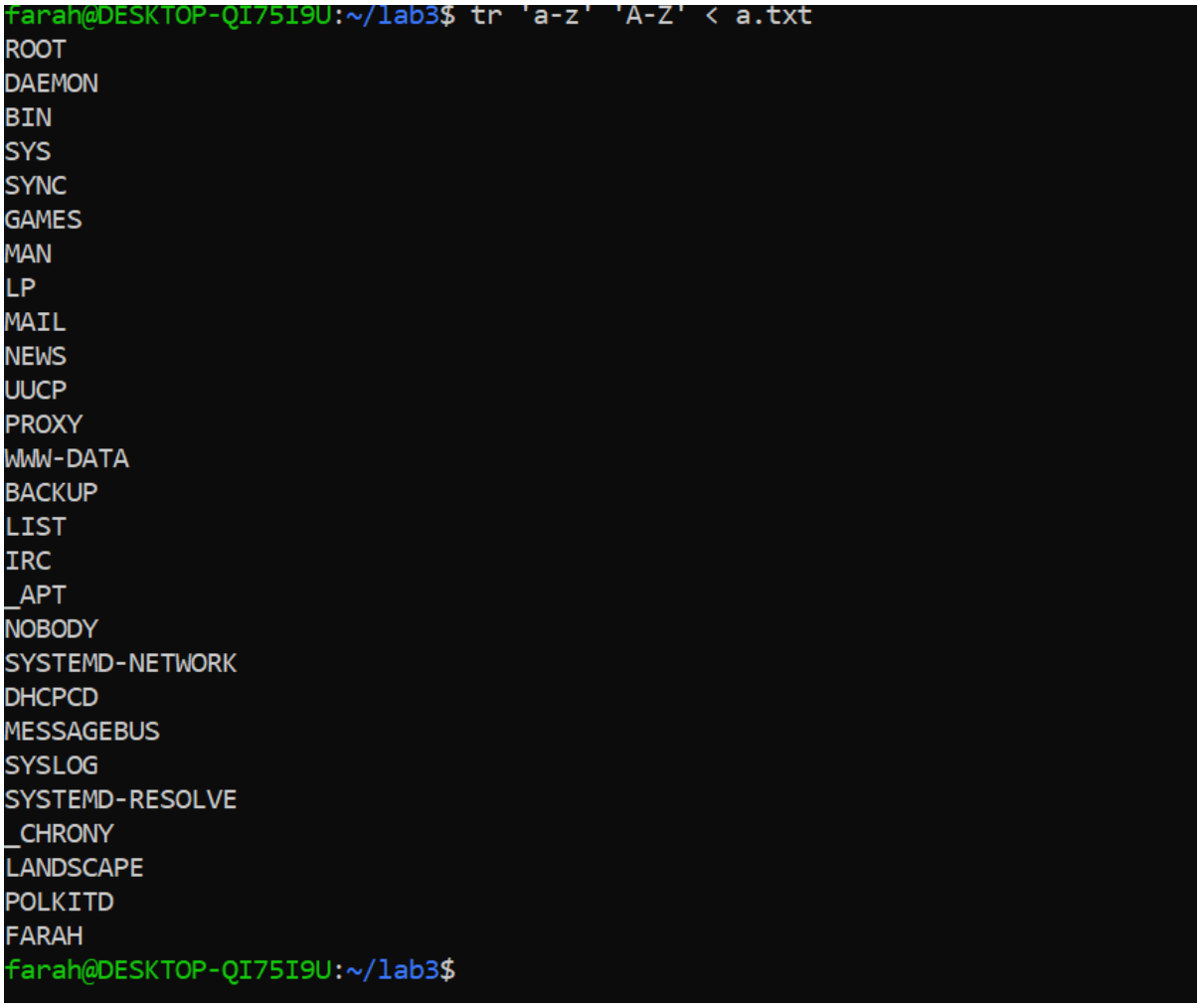
Description: The command `cut -d: -f1,7 pw.txt` extracts the username and shell fields from each passwd entry.

Screenshot 18: Combining files and translating characters

```
farah@DESKTOP-QI75I9U: ~/lab3
farah@DESKTOP-QI75I9U:~/lab3$ cut -d: -f7 pw.txt > b.txt
farah@DESKTOP-QI75I9U:~/lab3$ ls
a.txt  b.txt  dir1  pw.txt
farah@DESKTOP-QI75I9U:~/lab3$ paste a.txt b.txt
root    /bin/bash
daemon  /usr/sbin/nologin
bin     /usr/sbin/nologin
sys     /usr/sbin/nologin
sync    /bin/sync
games   /usr/sbin/nologin
man     /usr/sbin/nologin
lp      /usr/sbin/nologin
mail    /usr/sbin/nologin
news    /usr/sbin/nologin
uucp    /usr/sbin/nologin
proxy   /usr/sbin/nologin
www-data        /usr/sbin/nologin
backup  /usr/sbin/nologin
list    /usr/sbin/nologin
irc     /usr/sbin/nologin
_apt    /usr/sbin/nologin
nobody  /usr/sbin/nologin
systemd-network /usr/sbin/nologin
dhcpcd  /bin/false
messagebus      /usr/sbin/nologin
syslog  /usr/sbin/nologin
systemd-resolve /usr/sbin/nologin
_chrony /usr/sbin/nologin
landscape       /usr/sbin/nologin
polkitd /usr/sbin/nologin
farah   /bin/bash
farah@DESKTOP-QI75I9U:~/lab3$ echo "hello" | tr 'l' 'L'
heLLo
```

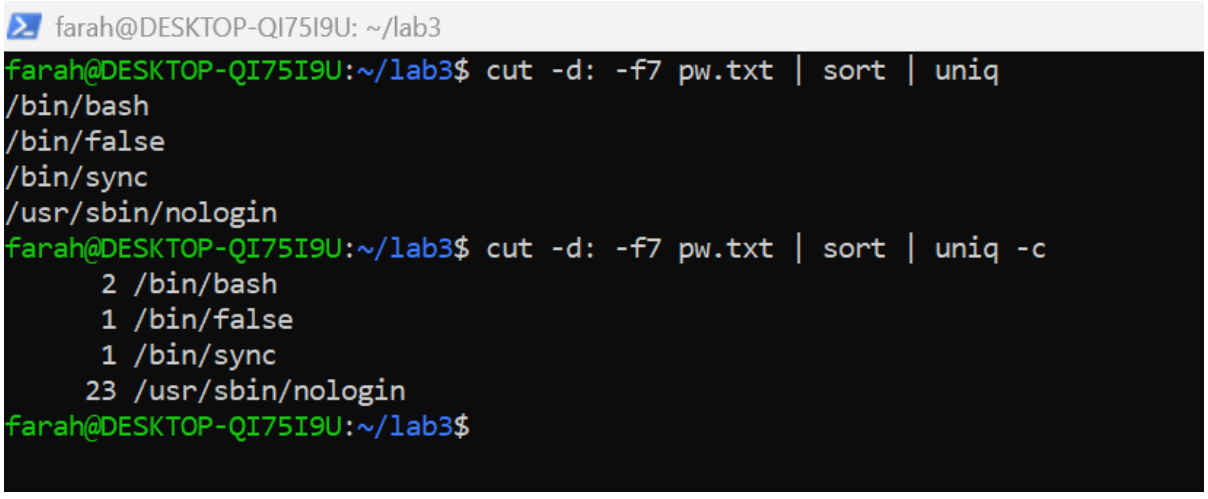
Description: The screenshot demonstrates `paste` to combine file contents and `tr 'a-z' 'A-Z'` to translate lowercase letters to uppercase.

Screenshot 19: Extracting and sorting shell fields



Description: The command pipeline extracts field 7 from `pw.txt` and passes it through `sort` and `uniq` to obtain distinct shell values.

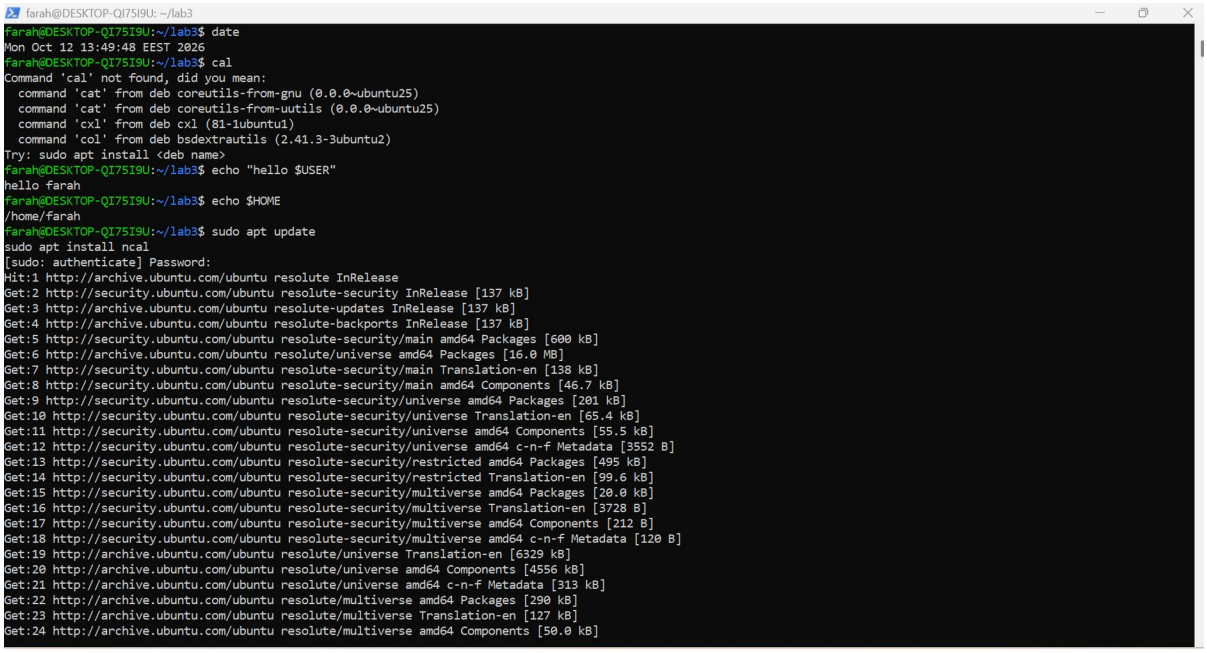
Screenshot 20: Counting unique values with uniq -c



Description: The pipeline is extended with `uniq -c` to display each unique shell value together with the number of occurrences.

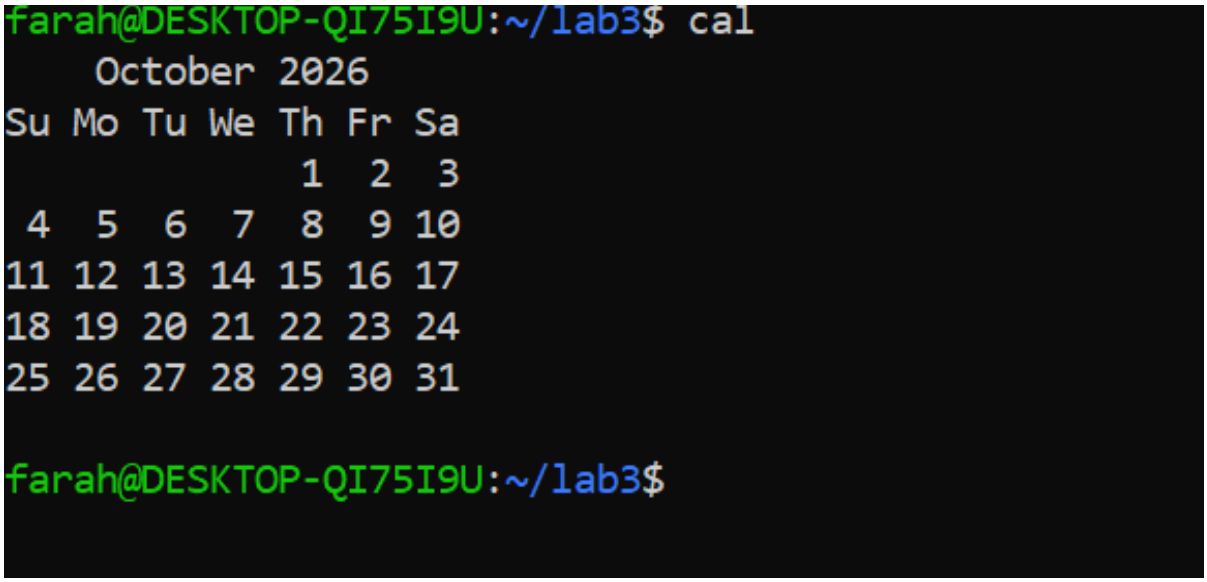
4. AI Agent – Environment Setup

Screenshot 21: Updating Ubuntu package information



Description: This setup screenshot shows `sudo apt update` downloading current package metadata from the Ubuntu repositories before installing the tools needed for the AI-agent environment.

Screenshot 22: Displaying the calendar with cal



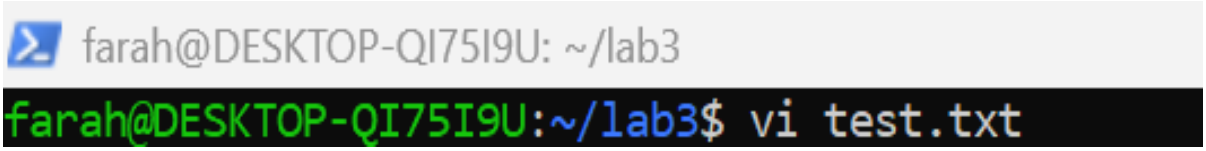
Description: The `cal` command displays the October 2026 calendar in the terminal.

Screenshot 23: Opening vi



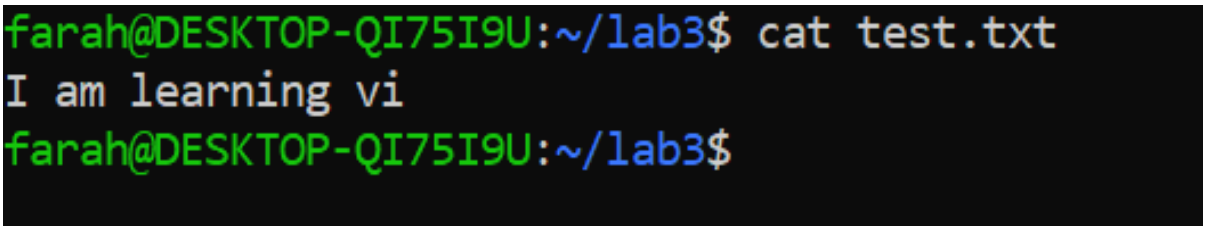
Description: The `vi` editor is opened in the lab directory, demonstrating the command used to create/edit a text file interactively.

Screenshot 24: Starting vi on test.txt



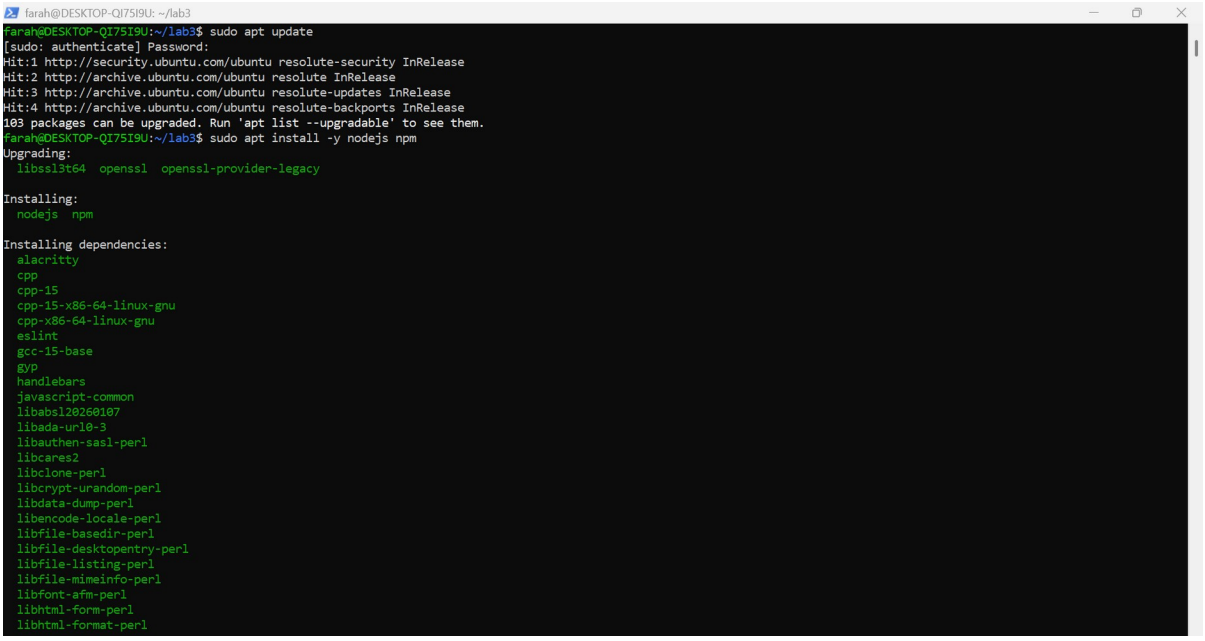
Description: The command `vi test.txt` opens the file in the vi editor for editing.

Screenshot 25: Verifying the edited file with cat



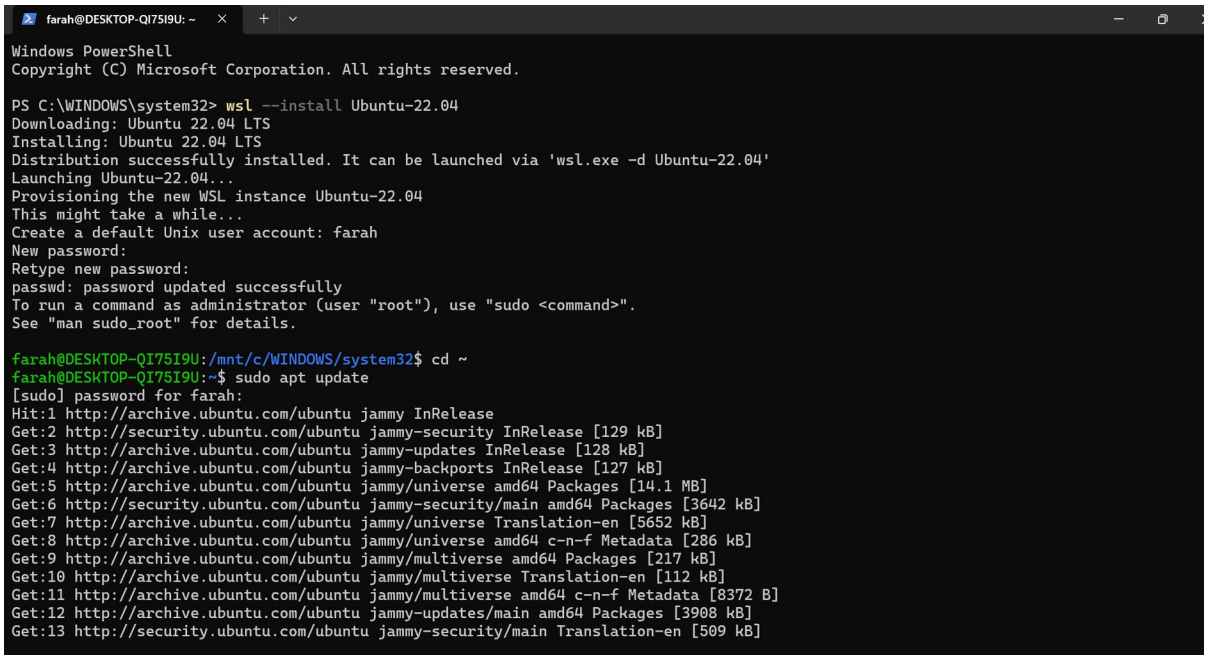
Description: The file is displayed with `cat test.txt`; the output confirms the text `I am learning vi`.

Screenshot 26: Installing Linux prerequisites



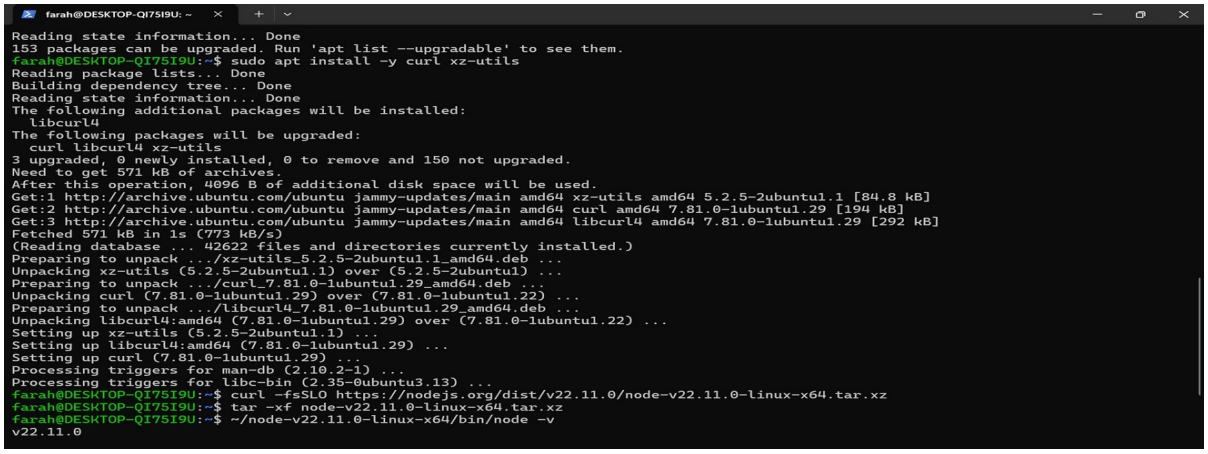
Description: The screenshot shows package installation for the command-line environment, including tools required for downloading and handling the Node.js installation.

Screenshot 27: Installing Ubuntu 22.04 through WSL



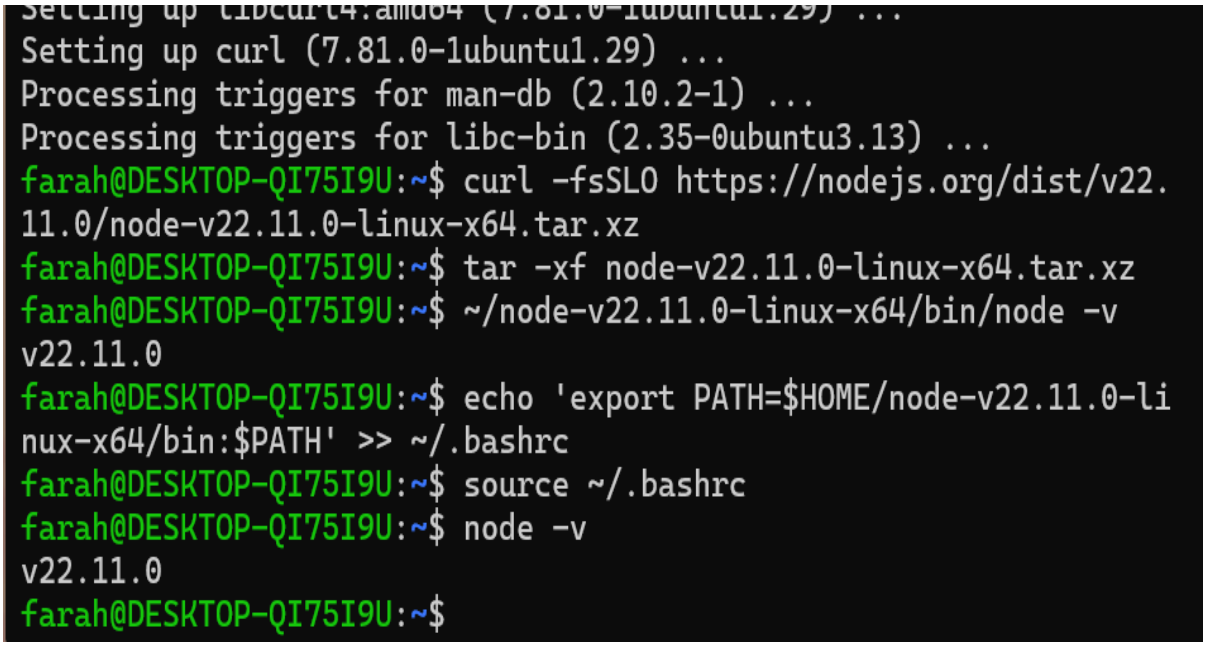
Description: Windows PowerShell runs `wsl --install Ubuntu-22.04`. The output confirms that Ubuntu 22.04 LTS is installed and a default Unix user named `farah` is created.

Screenshot 28: Installing and verifying Gemini CLI



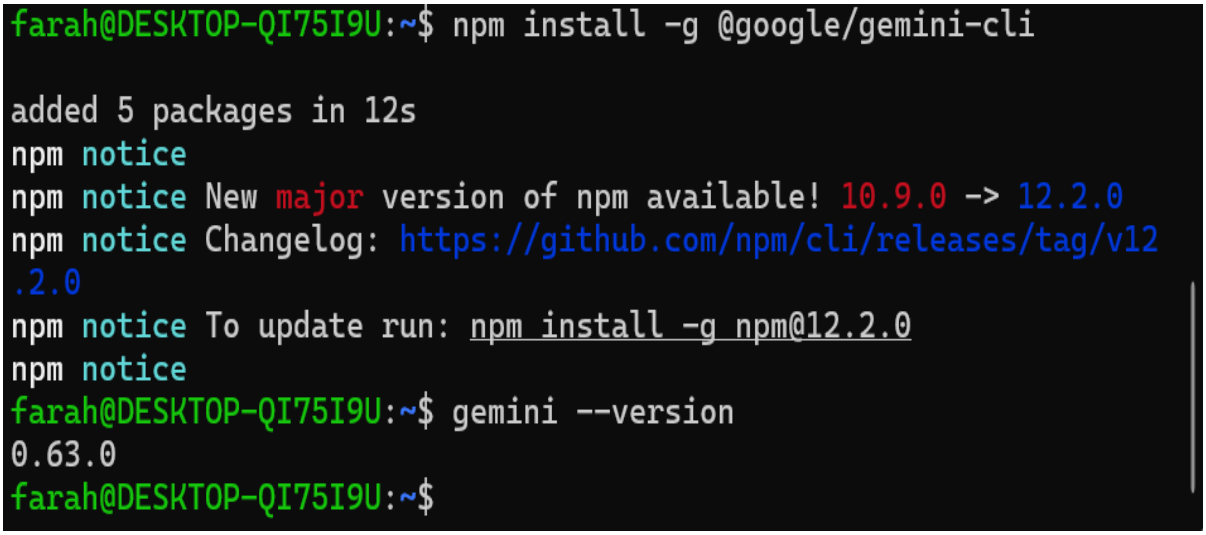
Description: The command `npm install -g @google/gemini-cli` installs the Gemini CLI. The following `gemini --version` check reports version 0.63.0.

Screenshot 29: Installing Node.js



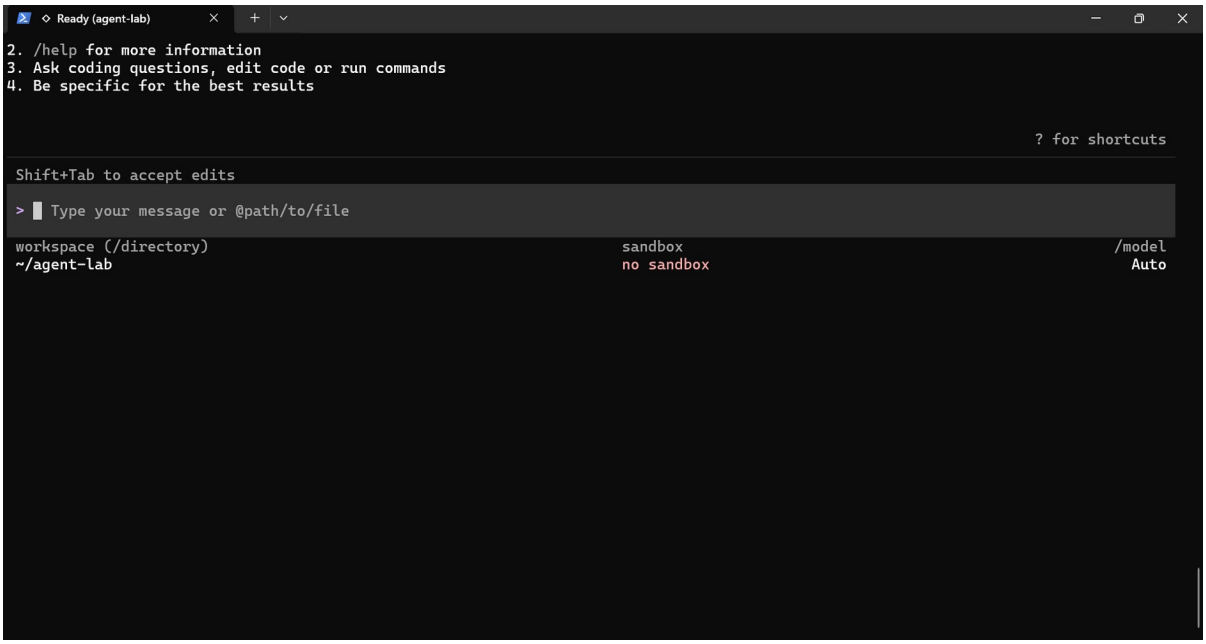
Description: The screenshot shows the Node.js installation process, including extraction/setup of Node.js v22.11.0 and related package configuration.

Screenshot 30: Configuring and verifying Node.js



Description: The PATH is updated to include Node.js v22.11.0, the shell configuration is reloaded, and `node -v` confirms the installed version.

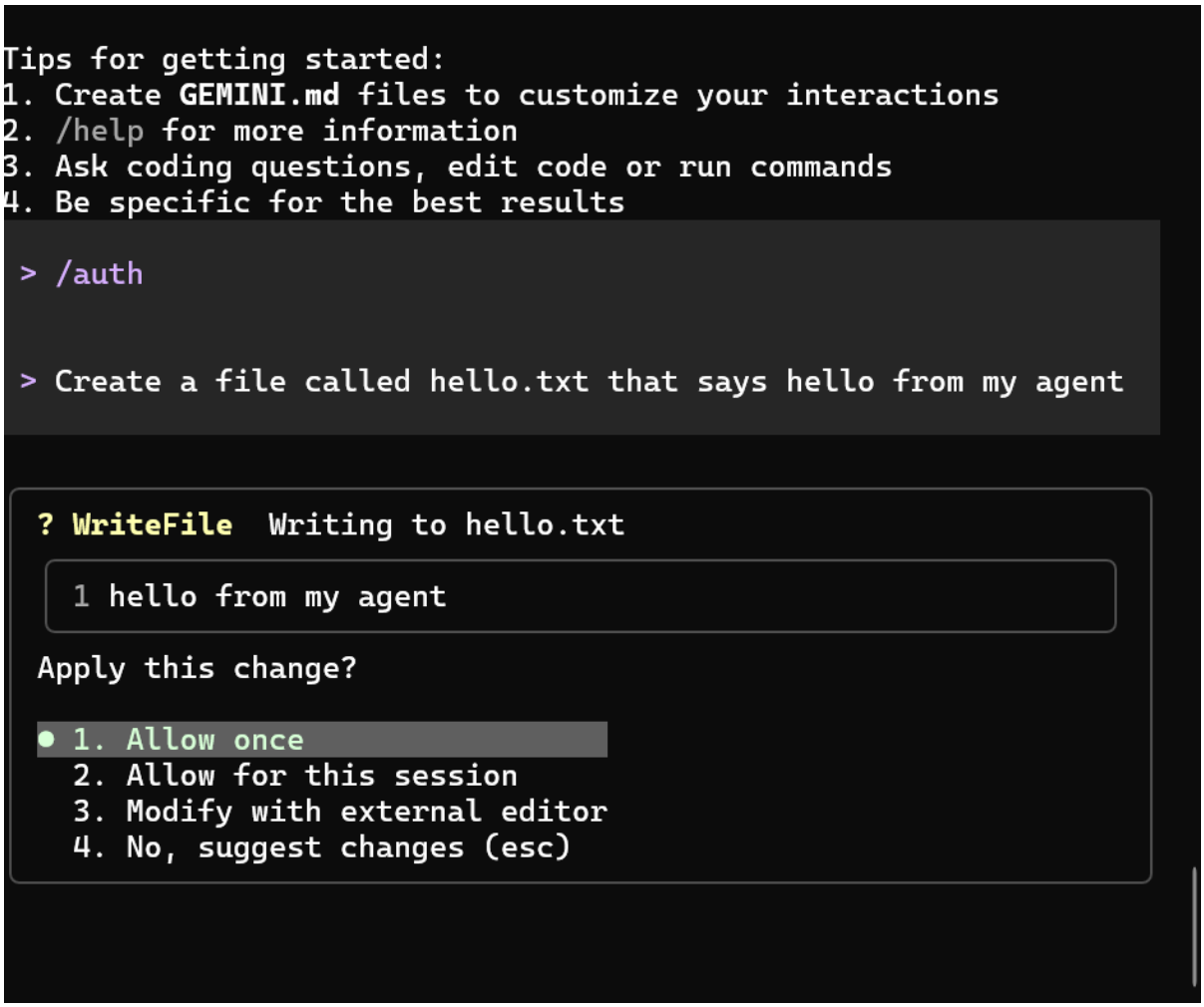
Screenshot 31: Launching the AI agent workspace



Description: The Gemini CLI interface is opened and the workspace is set to `~/agent-lab`. The screen provides the agent's interaction options and working-directory context.

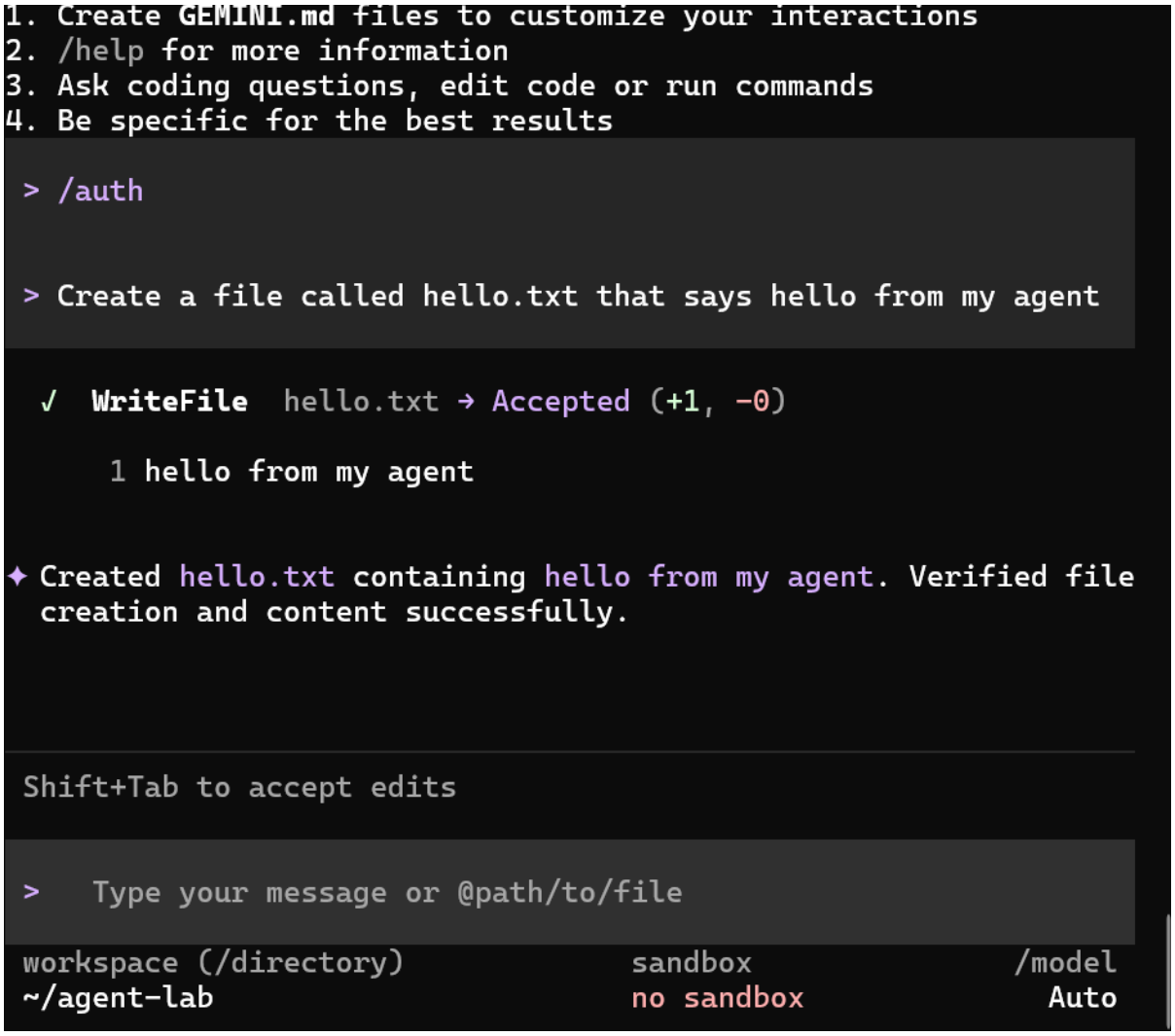
5. AI Agent – Experiment 1: File Creation and Verification

Screenshot 32: Giving the AI agent its first task



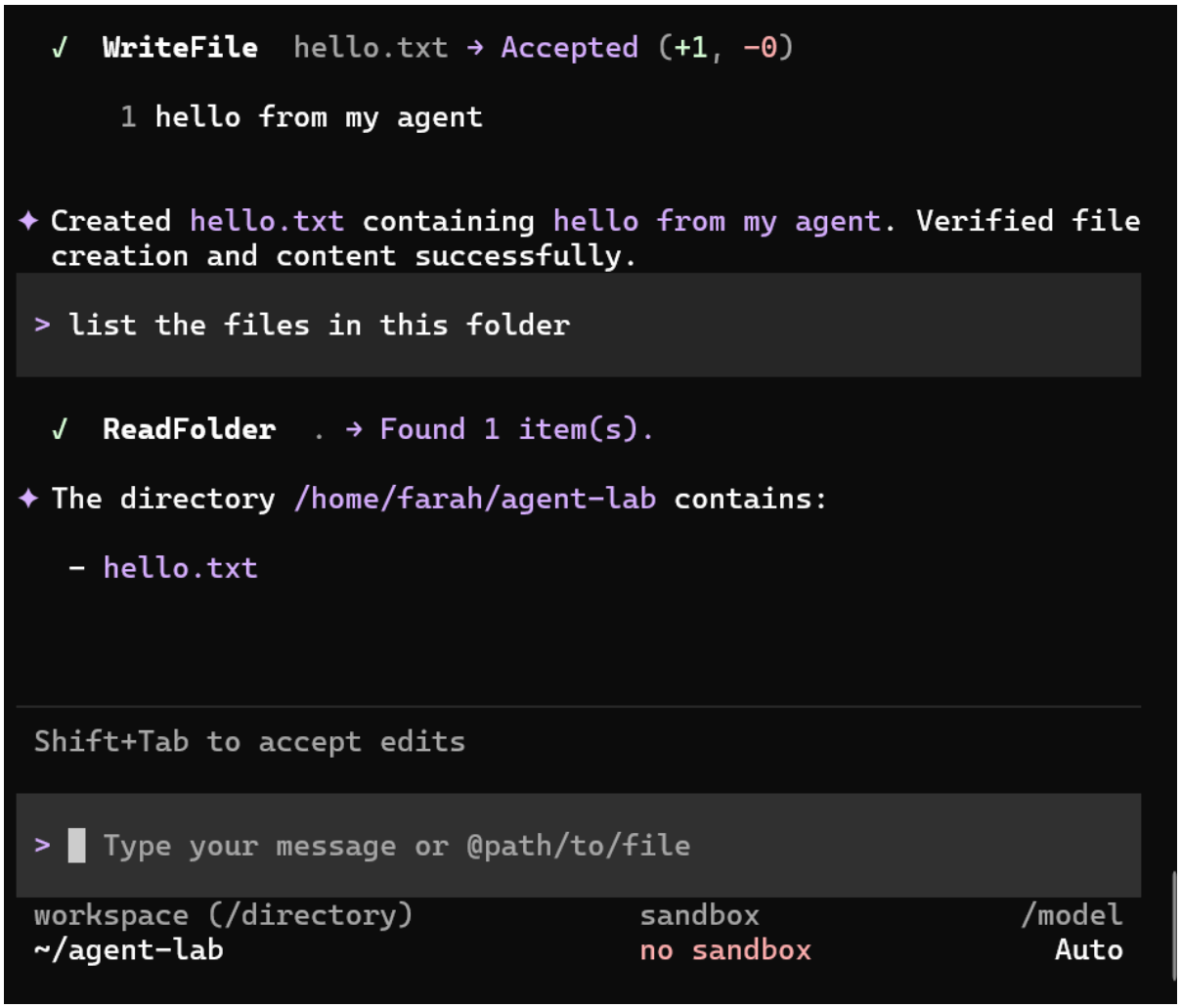
Description: The natural-language request asks the agent to create `hello.txt` containing `hello from my agent`. The agent proposes a `WriteFile` action and requests approval before changing the filesystem.

Screenshot 33: AI agent creates hello.txt



Description: After approval, the agent writes the requested content to `hello.txt` and reports that the file was created successfully.

Screenshot 34: AI agent verifies the created file



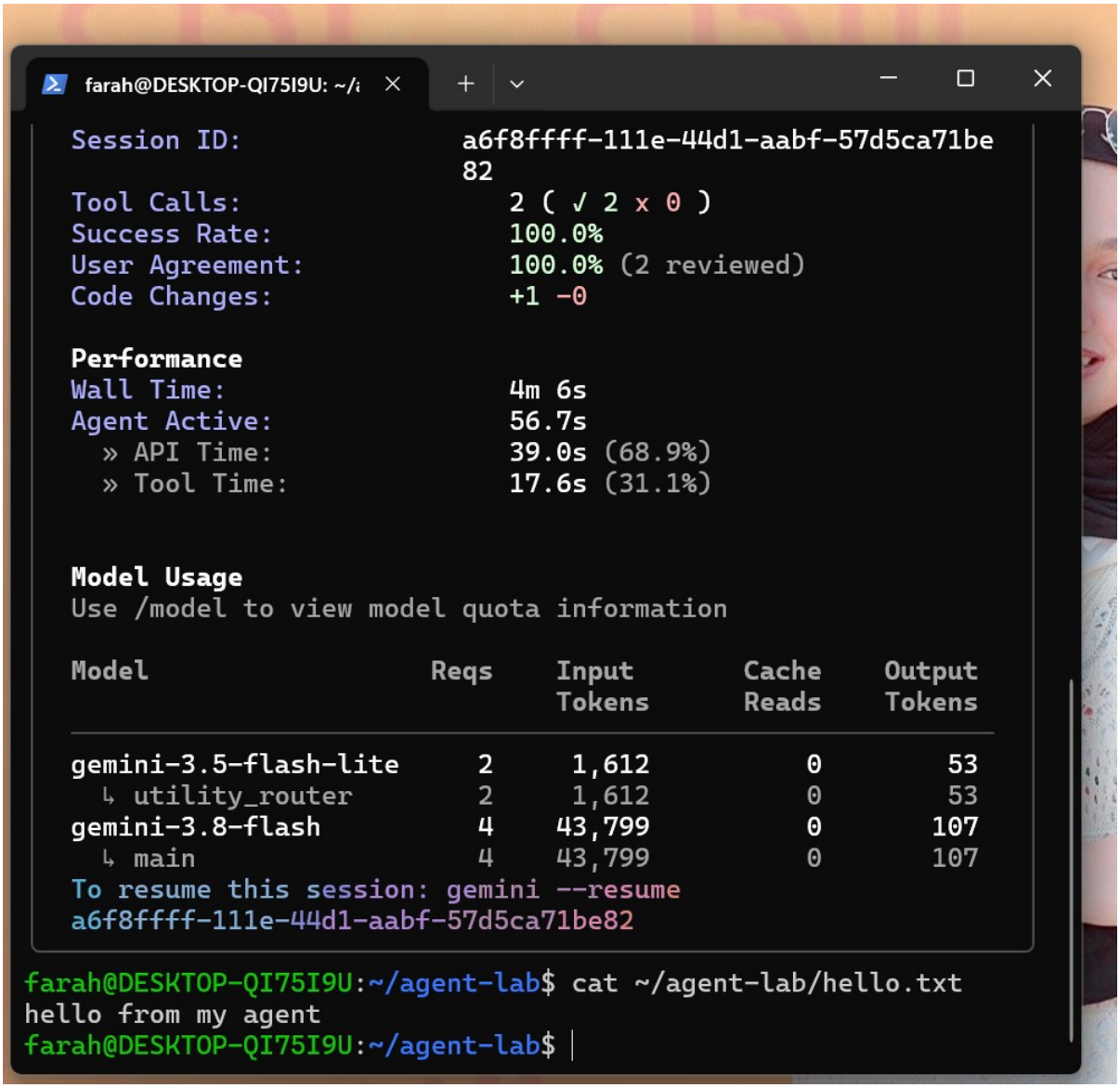
Description: The agent lists the workspace and confirms that `hello.txt` exists, demonstrating post-action verification.

Screenshot 35: Empty/transition screenshot



Description: This supplied screenshot contains only a small dash and no readable command or result. It is retained in the report so that no supplied screenshot is omitted.

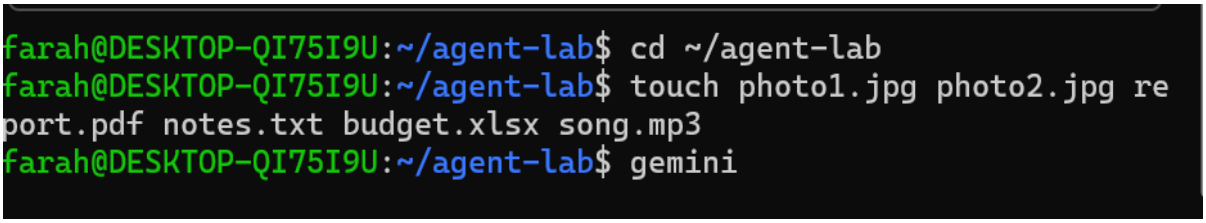
Screenshot 36: AI session statistics and file verification



Description: The screenshot displays the agent session statistics, including successful tool calls and user agreement, followed by `cat ~/agent-lab/hello.txt`, which prints `hello from my agent`.

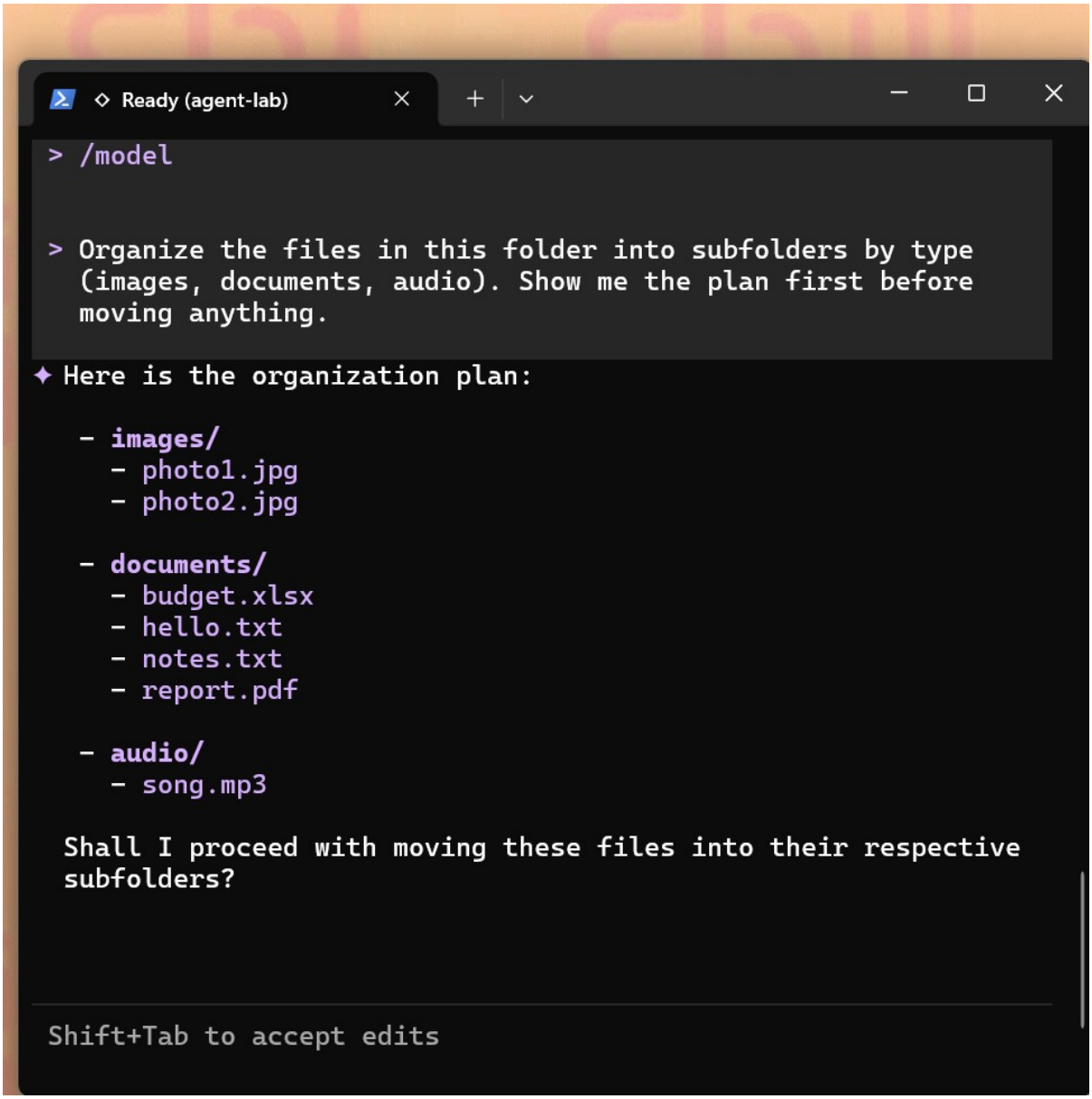
6. AI Agent – Experiment 2: Automated File Organization

Screenshot 37: Creating sample files for the creative task



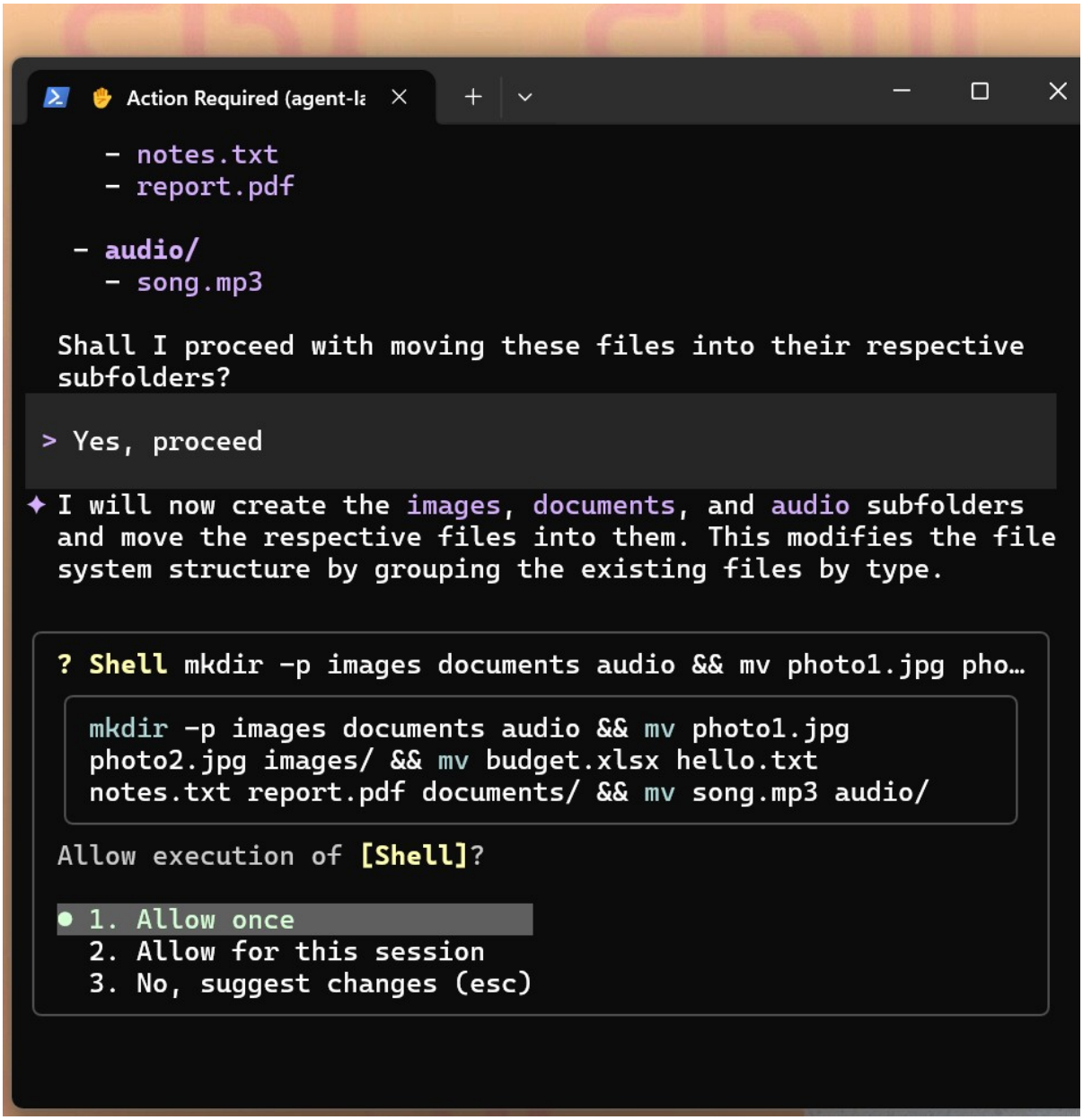
Description: The shell creates multiple files with different extensions—JPG images, a PDF, a TXT file, an XLSX workbook and an MP3—inside `~/agent-lab`, then launches Gemini.

Screenshot 38: AI agent proposes an organization plan



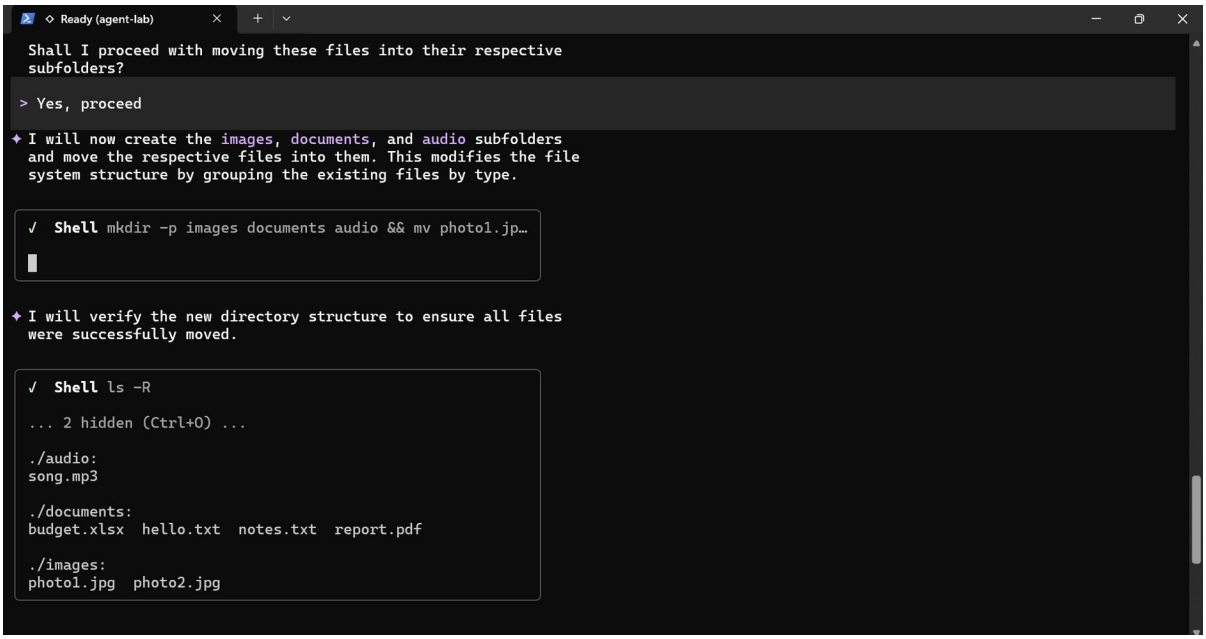
Description: The agent interprets the request to organize the files by type and proposes `images`, `documents`, and `audio` subfolders with the corresponding files.

Screenshot 39: Approval before executing shell commands



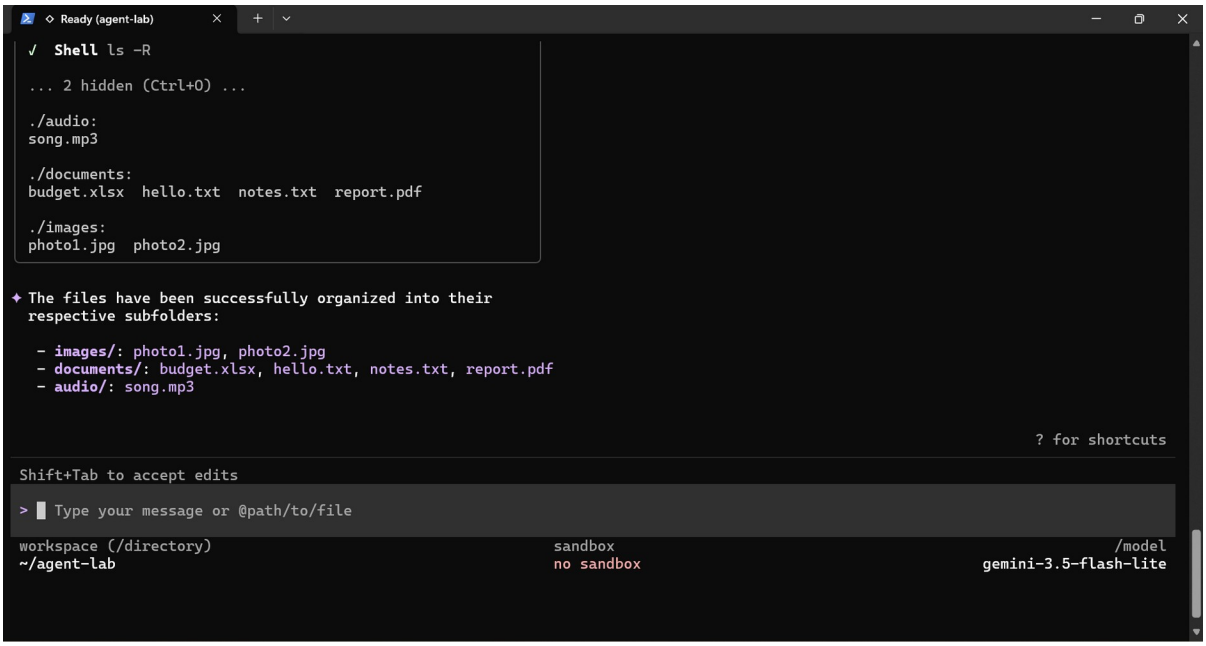
Description: The agent asks for confirmation before running the shell command that creates the folders and moves each file into its respective directory.

Screenshot 40: AI agent executes the organization plan



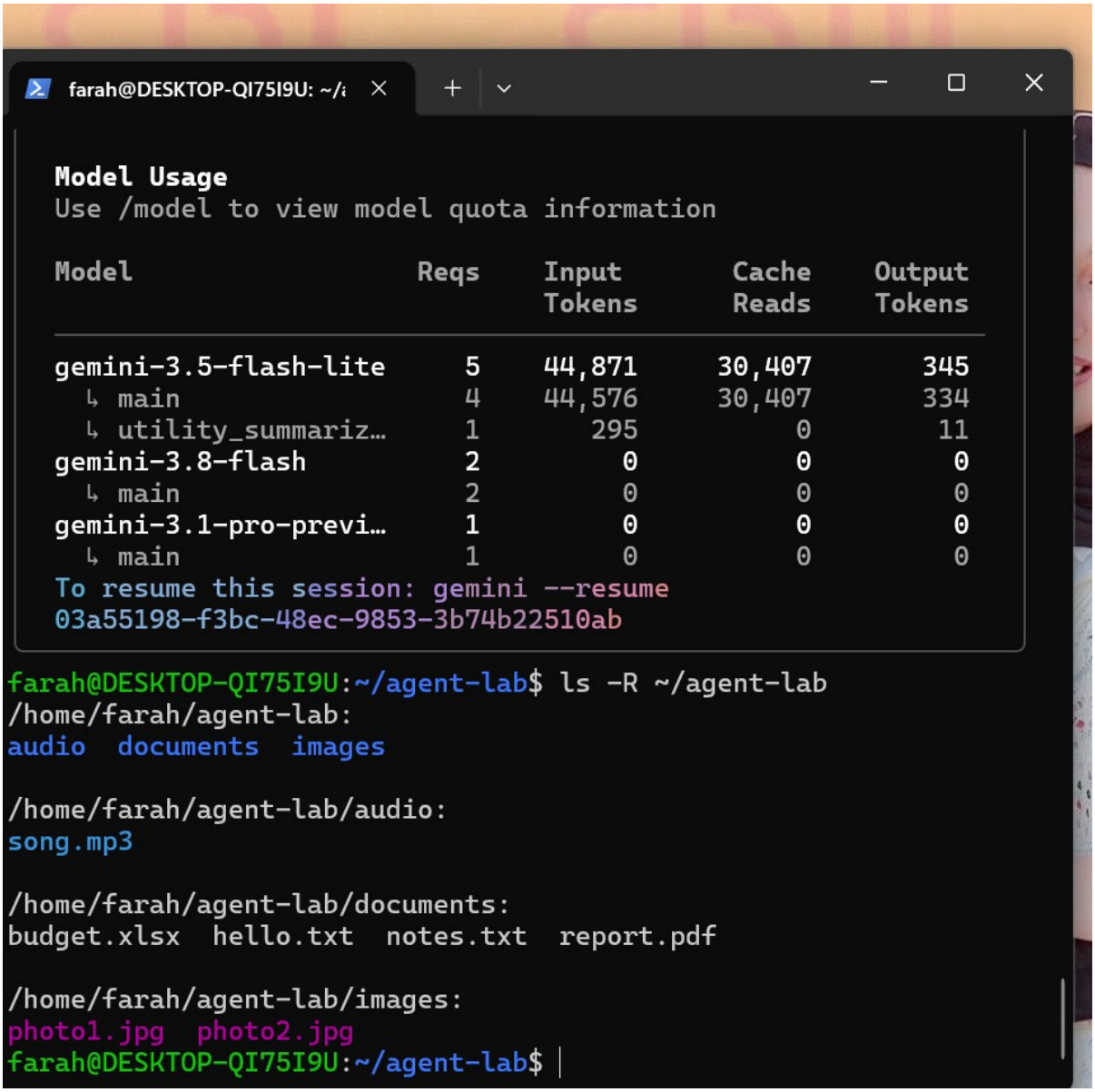
Description: The agent runs the approved shell operation and then checks the directory structure, confirming that the files were moved into the planned categories.

Screenshot 41: Final organized directory structure



Description: The agent reports that the files were successfully organized into images, documents and audio, and shows the resulting structure.

Screenshot 42: Final AI model/session usage



Description: The final screenshot shows the model-usage/session statistics and the resulting directories/files in `~/agent-lab`, providing evidence of the completed autonomous task.

7. Reflection

The practical work provided direct experience with Linux command-line administration rather than only theoretical command syntax. The screenshots show navigation, file creation, copying, moving, deleting, viewing, sorting and searching data, as well as using vi for text editing. The AI-agent part extended the exercise from manually typing every command to giving the agent a goal in natural language and reviewing the actions it proposed.

The first AI experiment demonstrated a controlled workflow: the agent interpreted the request, proposed a file-writing action, asked for approval, executed the operation and verified the result. The second experiment was more creative: several files with different extensions were created and the agent planned and executed an organization scheme using shell commands. This showed how an

AI agent can assist with repetitive administration tasks while still keeping the user in control of filesystem-changing actions.

8. Conclusion

The submitted evidence demonstrates both required parts of the task. The Lecture 3 commands were practiced and documented, and the AI agent was used to generate and execute Linux commands for practical tasks. The experiments also reinforce an important network administration practice: AI-generated commands should be reviewed before approval, especially when they modify files or system state.